Multi-City Environmental Health Analysis — Process & Execution Guide

1. Data Ingestion & API Extraction (Python)

Extracted multi-city historical telemetry including temperature, relative humidity, $\text{PM}_{2.5}$, PM_{10}, NO_2, and Ozone.

Python script sending API requests to Open-Meteo endpoints.

```python
import requests
import pandas as pd

cities = {
    "Delhi": {"lat": 28.6139, "lon": 77.2090},
    "Mumbai": {"lat": 19.0760, "lon": 72.8777},
    "Bengaluru": {"lat": 12.9716, "lon": 77.5946},
    "Kolkata": {"lat": 22.5726, "lon": 88.3639},
    "Chennai": {"lat": 13.0827, "lon": 80.2707},
    "Hyderabad": {"lat": 17.3850, "lon": 78.4867},
    "Ahmedabad": {"lat": 23.0225, "lon": 72.5714},
    "Pune": {"lat": 18.5204, "lon": 73.8567},
    "Jaipur": {"lat": 26.9124, "lon": 75.7873},
    "Gulbarga": {"lat": 17.3376, "lon": 76.8379}
}

all_city_data = []

for city_name, coords in cities.items():
    weather_url = (
        f"https://api.open-meteo.com/v1/forecast?"
        f"latitude={coords['lat']}&longitude={coords['lon']}&"
        f"hourly=temperature_2m,relative_humidity_2m,precipitation,"
        f"wind_speed_10m,surface_pressure&past_days=30"
    )
    aq_url = (
        f"https://air-quality-api.open-meteo.com/v1/air-quality?"
        f"latitude={coords['lat']}&longitude={coords['lon']}&"
        f"hourly=pm2_5,pm10,nitrogen_dioxide,ozone&past_days=30"
    )

    w_res = requests.get(weather_url).json()
    aq_res = requests.get(aq_url).json()
```

Pandas Data Wrangling & Cleaning

Normalized raw JSON responses, handled missing/null values, reformatted timestamps, and cleaned data types across all telemetry metrics.

Pandas operations for data sanitization, null value handling, and type conversion.

```python
import pandas as pd
import numpy as np

RAW_PATH = "raw_air_weather_data.csv"
CLEAN_PATH = "output/clean_air_weather_data.csv"


# 1. LOAD

df = pd.read_csv(RAW_PATH)
print(f"[1] Loaded raw data: {df.shape[0]} rows, {df.shape[1]} cols")


# 2. STANDARDIZE COLUMN NAMES & TEXT FIELDS

df.columns = (
    df.columns.str.strip()
    .str.lower()
    .str.replace(" ", "_")
)

# Clean city names: strip whitespace, fix casing, collapse internal spaces
df["city"] = (
    df["city"]
    .astype(str)
    .str.strip()
    .str.replace(r"\s+", " ", regex=True)
    .str.title()
)

# Catch near-duplicate city spellings (e.g. "Bangalore" vs "Bengaluru")
city_alias_map = {
    "Bangalore": "Bengaluru",
    "Calcutta": "Kolkata",
    "Madras": "Chennai",
    "Bombay": "Mumbai",
}
df["city"] = df["city"].replace(city_alias_map)
```

(parameter) regex: bool

```python
print("[2] Cities found:", sorted(df["city"].unique()))


# 3. FIX DTYPES

df["timestamp"] = pd.to_datetime(df["timestamp"], errors="coerce")

numeric_cols = [
    "temperature_2m", "relative_humidity_2m", "precipitation",
    "wind_speed_10m", "surface_pressure", "pm2_5", "pm10",
    "nitrogen_dioxide", "ozone",
]
for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors="coerce")

# Any rows where timestamp failed to parse are unusable -> drop
bad_ts = df["timestamp"].isna().sum()
if bad_ts:
    print(f"    Dropping {bad_ts} rows with unparseable timestamps")
    df = df.dropna(subset=["timestamp"])

print("[3] Dtypes fixed:")
print(df.dtypes)


# 4. HANDLE DUPLICATES

exact_dupes = df.duplicated().sum()
df = df.drop_duplicates()
print(f"[4] Dropped {exact_dupes} exact duplicate rows")

key_dupes = df.duplicated(subset=["city", "timestamp"]).sum()
if key_dupes:
    print(f"    Found {key_dupes} duplicate (city, timestamp) keys -> keeping last reading")
    df = df.drop_duplicates(subset=["city", "timestamp"], keep="last")
else:
    print("    No duplicate (city, timestamp) keys")

# 5. HANDLE MISSING VALUES

missing_summary = df.isna().sum()
```

```
# 5. HANDLE MISSING VALUES

missing_summary = df.isna().sum()
print("[5] Missing values per column:\n", missing_summary[missing_summary > 0] if missing_summary.any() else "    None found")

if df[numeric_cols].isna().any().any():
    df = df.sort_values(["city", "timestamp"])
    # Time-aware fill: interpolate within each city's own hourly series first,
    # then forward/backward fill any remaining edge gaps.
    df[numeric_cols] = (
        df.groupby("city")[numeric_cols]
        .transform(lambda s: s.interpolate(method="linear", limit_direction="both"))
    )
    # Fallback: if a whole city-column was NaN, fill with global column median
    for col in numeric_cols:
        if df[col].isna().any():
            df[col] = df[col].fillna(df[col].median())
    print("    Missing numeric values imputed via per-city linear interpolation")

# Drop rows still missing the essential keys
before = len(df)
df = df.dropna(subset=["city", "timestamp"])
if len(df) < before:
    print(f"    Dropped {before - len(df)} rows with missing city/timestamp")

# 6. VALIDATE / FIX OUT-OF-RANGE VALUES

4 references
def clip_report(series, low, high, name):
    n_bad = ((series < low) | (series > high)).sum()
    if n_bad:
        print(f"    {name}: {n_bad} values outside [{low}, {high}] -> clipped")
    return series.clip(lower=low, upper=high)

print("[6] Validating physical ranges")
df["relative_humidity_2m"] = clip_report(df["relative_humidity_2m"], 0, 100, "relative_humidity_2m")
for col in ["precipitation", "wind_speed_10m", "pm2_5", "pm10", "nitrogen_dioxide", "ozone"]:
    df[col] = clip_report(df[col], 0, df[col].max(), col)  # no negative pollutant/weather readings

# Surface pressure: sane range for sea-level-adjusted station pressure (hPa)
df["surface_pressure"] = clip_report(df["surface_pressure"], 850, 1085, "surface_pressure")
```

```
# 7. CHECK FOR MISSING HOURLY TIMESTAMPS (GAPS) PER CITY

print("[7] Checking hourly continuity per city")
gap_rows = []
for city, g in df.groupby("city"):
    g = g.sort_values("timestamp")
    full_range = pd.date_range(g["timestamp"].min(), g["timestamp"].max(), freq="h")
    missing_hours = full_range.difference(g["timestamp"])
    if len(missing_hours):
        print(f"    {city}: {len(missing_hours)} missing hour(s)")
        gap_rows.append(
            pd.DataFrame({"city": city, "timestamp": missing_hours})
        )

if gap_rows:
    gaps_df = pd.concat(gap_rows, ignore_index=True)
    df = pd.concat([df, gaps_df], ignore_index=True)
    df = df.sort_values(["city", "timestamp"])
    df[numeric_cols] = (
        df.groupby("city")[numeric_cols]
        .transform(lambda s: s.interpolate(method="linear", limit_direction="both"))
    )
    print(f"    Inserted {len(gaps_df)} missing hourly rows and interpolated their values")
else:
    print("    No gaps found — every city has a complete hourly series")

# 8. FINAL SORT, RESET INDEX, SCHEMA CHECK
```

2. SQL Data Storage & Query Validation

Schema Creation & Data Loading

Defined database table schemas and loaded sanitized Pandas DataFrames into SQL.

SQL schema setup and table creation scripts.

```python
# 9. EXPORT & LOAD BACK TO MYSQL


CLEAN_PATH = r"C:\Users\adv danish\OneDrive\Desktop\OWN PROJECT\clean_air_weather_data.csv"
df.to_csv(CLEAN_PATH, index=False)
print(f"[9] Saved cleaned data -> {CLEAN_PATH}")

from sqlalchemy import create_engine
db_user = "root"
db_password = "password"  # Update with your password
db_host = "localhost"
db_port = "3306"
db_name = "air_quality_project"

engine = create_engine(f"mysql+pymysql://{db_user}:{db_password}@{db_host}:{db_port}/{db_name}")
df.to_sql("clean_air_weather", con=engine, if_exists="replace", index=False, chunksize=1000)
print("Successfully loaded cleaned data into MySQL table `clean_air_weather`!")
```

SQL Problem-Solving & Analytical Queries

Executed SQL analytical queries to isolate extreme pollution events, rank cities by peak particulate levels, and evaluate weather correlations prior to Power BI visualization.

SQL analytical queries evaluating city-level pollution spikes and environmental trends.

1. Baseline Overview & Averages Queries basic schema, loads raw telemetry, and ranks cities by overall average $\text{PM}_{2.5}$ concentration to establish baseline pollution levels.

city	avg_pm25
Delhi	63.42
Jaipur	34.08
Kolkata	21.42
Ahmedabad	19.65
Chennai	17.86
Mumbai	14.52
Hyderabad	12.14
Gulbarga	8.23
Bengaluru	7.48
Pune	6.29

2. Weekday vs. Weekend Analysis

Compares average $\text{PM}_{2.5}$ across weekdays and weekends using DAYOFWEEK() to evaluate the impact of commuter traffic and industrial operations.

city	day_type	avg_pm25
Ahmedabad	Weekday	20.01
Ahmedabad	Weekend	18.73
Bengaluru	Weekday	7.49
Bengaluru	Weekend	7.44
Chennai	Weekday	18.23
Chennai	Weekend	16.95
Delhi	Weekday	61.4
Delhi	Weekend	68.49
Gulbarga	Weekday	8.34
Gulbarga	Weekend	7.96
Hyderabad	Weekday	12.48
Hyderabad	Weekend	11.3
Jaipur	Weekday	34.92
Jaipur	Weekend	31.99
Kolkata	Weekday	23.14
Kolkata	Weekend	17.12
Mumbai	Weekday	14.97
Mumbai	Weekend	13.38
Pune	Weekday	6.49
Pune	Weekend	5.78

3. Daytime vs. Nighttime Trapping Aggregates fine particulate matter into 12-hour day/night buckets to detect pollution spikes caused by atmospheric boundary layer trapping.

city	time_of_day	avg_pm25
Ahmedabad	Daytime (6 AM – 6 PM)	23.62
Ahmedabad	Nighttime (6 PM – 6 AM)	14.96
Bengaluru	Daytime (6 AM – 6 PM)	9.16
Bengaluru	Nighttime (6 PM – 6 AM)	5.49
Chennai	Daytime (6 AM – 6 PM)	19.78
Chennai	Nighttime (6 PM – 6 AM)	15.59
Delhi	Nighttime (6 PM – 6 AM)	63.63
Delhi	Daytime (6 AM – 6 PM)	63.25
Gulbarga	Daytime (6 AM – 6 PM)	10.16
Gulbarga	Nighttime (6 PM – 6 AM)	5.96
Hyderabad	Daytime (6 AM – 6 PM)	13.83
Hyderabad	Nighttime (6 PM – 6 AM)	10.15
Jaipur	Nighttime (6 PM – 6 AM)	34.56
Jaipur	Daytime (6 AM – 6 PM)	33.68
Kolkata	Daytime (6 AM – 6 PM)	22.06
Kolkata	Nighttime (6 PM – 6 AM)	20.68
Mumbai	Daytime (6 AM – 6 PM)	15.55
Mumbai	Nighttime (6 PM – 6 AM)	13.3
Pune	Daytime (6 AM – 6 PM)	8.19
Pune	Nighttime (6 PM – 6 AM)	4.04

4. Thermal Range Extremes Identifies maximum and minimum temperatures (`temperature_2m`) per city to establish regional weather limits.

city	max_temp	min_temp
Ahmedabad	35.2	25.1
Bengaluru	30.7	19.3
Chennai	36.6	25.2
Delhi	34.9	24.6
Gulbarga	31.5	22.8
Hyderabad	32	22.5
Jaipur	34.2	23.8
Kolkata	33.4	25
Mumbai	29.7	24.7
Pune	29.1	21.8

5. Rush Hour Multi-Metric Traffic Analysis

Segments hourly telemetry into morning rush, evening rush, late-night, and off-peak windows to evaluate how daily traffic and environmental cycles impact pm2_5, temperature, humidity, and concentrations simultaneously.

city	time_period	avg_pm25	avg_temperature	avg_humidity	avg_no2
Ahmedabad	1. Morning Rush (7–10 AM)	20.36	32.18	56.82	2.52
Ahmedabad	2. Evening Rush (5–9 PM)	20.75	27.78	79.25	13.13
Ahmedabad	3. Late Night	13.39	26.98	82.30	8.29
Ahmedabad	4. Midday / Off-Peak	24.07	30.62	65.05	11.82
Bengaluru	1. Morning Rush (7–10 AM)	7.3	27.63	58.16	4.14
Bengaluru	2. Evening Rush (5–9 PM)	7.37	21.37	89.09	14.11
Bengaluru	3. Late Night	5.1	21.44	87.20	8.11
Bengaluru	4. Midday / Off-Peak	9.72	24.83	71.28	15.24
Chennai	1. Morning Rush (7–10 AM)	18.52	34.07	51.39	4.37
Chennai	2. Evening Rush (5–9 PM)	18.31	28.34	80.45	12.94
Chennai	3. Late Night	15.07	28.66	73.92	15.01
Chennai	4. Midday / Off-Peak	19.7	31.16	67.49	9.12
Delhi	1. Morning Rush (7–10 AM)	53.02	31.69	67.97	10.93
Delhi	2. Evening Rush (5–9 PM)	79.68	27.85	89.49	63.94
Delhi	3. Late Night	59.58	27.56	87.92	35.24
Delhi	4. Midday / Off-Peak	61.84	30.01	78.44	38.47
Gulbarga	1. Morning Rush (7–10 AM)	8.23	29.96	58.05	1.51
Gulbarga	2. Evening Rush (5–9 PM)	8.39	24.87	80.49	8.56
Gulbarga	3. Late Night	5.38	24.54	81.07	5.53
Gulbarga	4. Midday / Off-Peak	10.64	27.6	69.30	6.57
Hyderabad	1. Morning Rush (7–10 AM)	11.03	29.34	56.51	2.49
Hyderabad	2. Evening Rush (5–9 PM)	14.12	24.92	81.00	12.09
Hyderabad	3. Late Night	9.09	24.58	80.46	7.53
Hyderabad	4. Midday / Off-Peak	14.14	27.51	67.66	9.88
Jaipur	1. Morning Rush (7–10 AM)	32.06	31.18	62.87	2.28
Jaipur	2. Evening Rush (5–9 PM)	34.7	26.75	84.45	20.61
Jaipur	3. Late Night	34.44	26.44	83.24	11.1
Jaipur	4. Midday / Off-Peak	34.39	29.32	73.04	13.23
Kolkata	1. Morning Rush (7–10 AM)	18.95	30.21	79.71	4.05

6. Calculates standard deviations Standard Deviation and averages for pm10 , temperature, relative humidity, and no2 across cities to isolate unpredictable environmental fluctuations rather than relying solely on overall means.

city	avg_pm10	pm10_volatility	avg_temperature	temp_volatility	avg_humidity	humidity_volatility	avg_no2	no2_volatility
Delhi	127.18	133.88	29.13	2.3	81.76	12.69	38.24	25.45
Jaipur	100.62	77.69	28.26	2.3	76.70	13.06	12.32	9.67
Ahmedabad	42.07	25.55	29.23	2.46	71.67	13.93	9.51	7.07
Kolkata	23.62	11.44	28.61	1.74	87.28	7.86	9.86	6.45
Hyderabad	19.92	9.41	26.42	2.25	72.31	12.54	8.42	5.64
Mumbai	31.07	9.37	27.35	1.08	82.13	5.69	6.06	1.9
Gulbarga	14.85	8.68	26.54	2.4	73.19	11.73	5.84	3.71
Chennai	25.66	7.67	30.33	2.66	69.38	13.8	10.84	5.48
Pune	11.37	7.09	24.35	1.74	82.88	8.7	6.23	3.49
Bengaluru	11.84	6.05	23.59	2.89	77.44	15.2	11.07	7.23

8. Total Rain Events Count Filters records where `precipitation > 0` to quantify total hours of rain logged across cities.

city	total_rainy_hours
Mumbai	759
Kolkata	504
Bengaluru	329
Delhi	325
Pune	313
Gulbarga	289
Jaipur	244
Ahmedabad	233
Hyderabad	226
Chennai	205

9. Rainfall Severity Classification Categorizes precipitation hours into light drizzle ($\le 1.0\text{ mm}$) and heavy downpours ($> 1.0\text{ mm}$) using conditional aggregation.

city	light_drizzle_hours	heavy_rain_hours
Kolkata	380	124
Delhi	250	75
Mumbai	717	42
Bengaluru	292	37
Jaipur	213	31
Chennai	177	28
Gulbarga	263	26
Pune	288	25
Ahmedabad	219	14
Hyderabad	217	9

10. Peak Pollution Day Identification Uses Common Table Expressions (CTEs) and the `ROW_NUMBER()` window function to isolate the single worst air quality day per city.

city	reading_date	daily_pm25
Delhi	2026-08-31	116.19
Jaipur	2026-08-27	59.57
Kolkata	2026-08-28	48.75
Ahmedabad	2026-08-26	33.03
Chennai	2026-08-18	22.24
Mumbai	2026-08-25	20.47
Hyderabad	2026-08-26	19.12
Gulbarga	2026-08-20	13.13
Pune	2026-08-25	11.17
Bengaluru	2026-09-05	11.15